SHOPSPHERE AI

Product & Engineering

PRODUCT REQUIREMENTS DOCUMENT

ShopSphere AI

Personalized Shopping, Powered by Intelligence

Field	Detail
Document Title	Product Requirements Document — ShopSphere AI
Prepared For	Product & Engineering Leadership
Prepared By	Product Management, UX Design & Engineering Leadership
Document Version	1.0
Document Status	Approved for Implementation
Engagement Duration	4 Weeks (Full-Time, Single Contributor)
Infrastructure & Tooling Model	Fully Open-Source, Self-Hosted Development Stack
Classification	Internal — Confidential

Document Control

Version History

Version	Date	Author	Description of Change
0.1	Draft	Product Management Team	Initial draft scope and objectives circulated for internal review
0.5	Draft	UX Design Team	Added information architecture, module-level UX requirements, and state specifications
0.8	Review	Engineering Leadership	Added technical architecture, API layer design, recommendation engine specification
1.0	Final	Product, UX & Engineering	Approved for release to the ShopSphere AI engineering team

Document Approval

Role	Responsibility	Status
Product Manager	Scope, requirements, prioritization, acceptance criteria	Approved
UX Designer	Information architecture, interaction design, accessibility standards	Approved
Engineering Lead	Technical architecture, feasibility, technology stack, quality gates	Approved

Distribution

This document is distributed to the engineers assigned to the ShopSphere AI build. It serves as the single source of truth for scope, functional requirements, technical architecture, and evaluation criteria for the project. Engineers should treat this PRD the same way they would a specification handed to them on any product team — as the primary reference throughout the engagement, not a one-time read.

Table of Contents

1. Introduction 4
2. Resource & Tooling Policy 6
3. Product Overview 7
4. User Roles and Personas 8
5. Core Modules — Functional Requirements 9
6. AI Recommendation System 17
7. AI Shopping Assistant 20
8. Admin Dashboard 23
9. Frontend Architecture 25
10. API and Data Layer 27
11. Authentication 29
12. Performance Requirements 30
13. Security Requirements 31
14. Accessibility Requirements 32
15. Responsive Design Requirements 33
16. Loading, Error, and Empty States 34
17. Testing Strategy 35
18. Version Control and Collaboration Practice 36
19. Development Plan — 4-Week Delivery Schedule 37
20. Edge Cases and Expected Behavior 42
21. Test Data Requirements 44
22. Engineering Deliverables 45
23. Definition of Done 46
24. Evaluation Rubric 47
25. Platform Roadmap — Current and Future Architecture 48
Appendix A — Zero-Cost Technology Stack 49
26. Closing Notes 50

1. Introduction

1.1 Purpose of This Document

This Product Requirements Document (PRD) defines the complete scope, functional behavior, technical architecture, and quality expectations for ShopSphere AI — a personalized e-commerce web application. It is written the way a PRD is written for a real product team: it assumes the reader (the engineer) is capable, and it focuses on what the product must do, how it should feel to use, and how success will be measured — not on teaching frontend concepts from first principles.

This document is the primary reference for the duration of the engagement. Any change in scope, technology, or acceptance criteria must be reflected here before implementation begins on that item.

1.2 Background

This engagement follows a cohort-based engineering model in which contributors are assigned production-style projects that mirror the scope and rigor of early-stage feature work at a technology company. The assigned domain is e-commerce, and the product brief calls for a modern, multi-category storefront with intelligent personalization at its core. ShopSphere AI is an original product with its own name, branding, and scope, designed specifically for this engagement.

1.3 Product Vision

ShopSphere AI gives shoppers a storefront that feels like it understands them. Instead of a static catalogue, every customer sees a homepage, product page, and cart that responds to what they have actually looked at, searched for, and added — powered by a transparent, explainable recommendation layer rather than a black box. The long-term vision is a platform where a lightweight, rule-based personalization engine can later be swapped for a machine-learning or LLM-based provider without rewriting the frontend.

1.4 Goals and Non-Goals

1.4.1 Goals

- Deliver a production-quality, responsive storefront covering discovery, product detail, cart, checkout, order tracking, wishlist, and reviews.
- Deliver a believable, signal-driven recommendation system with multiple distinct strategies.
- Deliver a rule-based conversational shopping assistant that parses structured shopping intent from natural-language-style input.
- Deliver a frontend admin dashboard for catalogue, order, and recommendation oversight.
- Demonstrate professional frontend engineering practice: clean architecture, typed or well-documented code, automated testing, accessibility, and performance discipline.

1.4.2 Non-Goals

- Standing up dedicated backend, database, or payment-gateway infrastructure within this engagement.
- Training or fine-tuning a machine learning model.
- Integrating a paid large language model (LLM) API.
- Building native mobile applications.
- Integrating a managed, external identity provider for authentication.

1.5 Intended Audience

This document is written for the Engineer delivering the project, and for the product and engineering leadership who will review it. It assumes familiarity with modern React/Next.js development and does not re-explain fundamental web development concepts.

2. Resource & Tooling Policy

Zero-Cost Guarantee

No paid API keys, cloud credits, premium software subscriptions, paid backend infrastructure, or payment gateway accounts are required for this project. It has been deliberately designed so that it can be completed in full — including its AI-style personalization features — using only free and open-source tools, free-tier services, and local mock data. Purchasing any API, subscription, hosting plan, or premium tool is not a project requirement and provides no additional evaluation marks.

2.1 What Is Not Required

To remove any ambiguity, the following are not required for this project:

- Paid API keys of any kind, including OpenAI, Anthropic (Claude), Google Gemini, or any other commercial LLM provider
- Cloud service credits (AWS, Azure, GCP, or similar)
- Paid or managed database instances
- Paid hosting or deployment plans
- Paid SaaS subscriptions of any kind (design tools, analytics tools, testing tools, etc.)
- Payment gateway merchant accounts (Razorpay, Stripe, or similar)
- Enterprise software licenses
- Premium developer tools or IDE licenses
- Private or dedicated backend infrastructure

2.2 Design Implication

Every requirement in this PRD has been written to be satisfiable at zero incremental infrastructure cost using a free/open-source equivalent. Where a scaled system would use a paid service (a managed database, a payment gateway, an LLM API), this PRD specifies a free, locally hosted, or mock equivalent, and explicitly names the scaled-platform alternative in a dedicated 'Platform Roadmap' section (Section 25) so the engineering reasoning is not lost.

2.3 Evaluation Fairness

No marks are awarded, directly or indirectly, for the use of a paid resource anywhere in this project. A submission that uses only free resources and satisfies the acceptance criteria in this document is eligible for a full 100/100 score under the rubric defined in Section 19. Conversely, using a paid API or service does not compensate for missing functional or quality requirements.

3. Product Overview

3.1 Summary

ShopSphere AI is a modern, multi-category e-commerce web application focused on personalized shopping discovery. The product is built entirely on the frontend, backed by a public product API and a locally implemented recommendation and assistant layer. From a user's perspective, ShopSphere AI behaves like a serious, production-grade storefront: fast, responsive, personalized, and forgiving of errors.

3.2 Customer-Facing Capabilities

- Discover, search, filter, and sort a multi-category product catalogue
- View rich product detail pages with variants, ratings, and related items
- Compare products side by side
- Maintain a cart and a wishlist across a session
- Complete a streamlined checkout and view the full order lifecycle
- Submit and browse product reviews
- Receive recommendations driven by real browsing, cart, and wishlist signals
- Interact with a rule-based conversational shopping assistant

3.3 Administrative Capabilities

A frontend-only admin dashboard provides catalogue, order, customer, and recommendation-analytics visibility for internal operational use. Section 9 defines this in detail, including how write operations are represented at this stage of the platform's build-out.

3.4 What Makes This 'Advanced'

The complexity in this project is not in any single screen — it is in the recommendation and assistant architecture, the state and caching design, the breadth of edge cases handled gracefully, and the discipline of building this to production coding standards on a lean, cost-efficient stack. This is the difference between a tutorial clone and a genuine engineering deliverable.

4. User Roles and Personas

4.1 Customer

The primary user of ShopSphere AI. A visitor browsing anonymously or a registered, signed-in shopper.

Capability Area	Description
Discovery	Browse, search, filter, sort, paginate the catalogue
Product Interaction	View details, compare, add to cart, add to wishlist
Transactions	Checkout, order history, order tracking
Community	Submit and read reviews and ratings
Personalization	Receive recommendations, use the AI shopping assistant
Account	Register, log in, log out, manage profile and addresses

4.2 Administrator

An internal operations persona who needs visibility into catalogue health, order flow, and how well the recommendation engine is performing. In this phase of the build, the Administrator's write actions (adding a product, changing an order status) are handled in local application state or against the product API's write endpoints, and are clearly distinguished from the fully persisted data layer planned for the next build phase.

> **Data Write Policy**
> Any admin action that appears to 'save' or 'update' data (product edits, order status changes, inventory adjustments) should be clearly labeled in the UI to reflect its current persistence scope (session or local storage at this build stage), so the distinction from the eventual production data layer stays clear as the platform matures.

4.3 Guest vs. Registered Access Matrix

Feature	Guest	Registered Customer	Administrator
Browse & search catalogue	Yes	Yes	Yes
Add to cart	Yes (local)	Yes (persisted to account)	N/A
Wishlist	Limited (session only)	Yes	N/A
Checkout	Requires login	Yes	N/A
Order history / tracking	No	Yes	View all
Submit reviews	No	Yes	Moderate
AI shopping assistant	Yes	Yes	N/A
Admin dashboard	No	No	Yes

5. Core Modules — Functional Requirements

Each module below is specified with functional requirements, UX expectations, and representative acceptance criteria in Given/When/Then format. Requirement IDs follow the pattern MOD-XX-NNN for traceability across design, implementation, and QA.

5.1 Landing Page (MOD-01)

The landing page is the storefront's first impression and the primary surface for personalization. It must load fast, communicate the breadth of the catalogue, and immediately demonstrate relevance.

Required Sections

- Header with logo, primary navigation, search bar, cart icon with item count, wishlist icon, account menu
- Category navigation (mega-menu or horizontal category strip)
- Hero section featuring seasonal or promotional content
- Featured Products rail
- Trending Products rail (derived from mock popularity signals)
- Personalized Recommendations rail ('Recommended For You')
- Recently Viewed rail (hidden when history is empty)
- AI Shopping Assistant entry point (floating action button or docked panel)
- Footer with sitemap links, policy pages, and newsletter signup (non-functional simulation acceptable)

MOD-01-001 — First-time visitor homepage

Condition	Description
Given	a visitor with no browsing history opens ShopSphere AI for the first time
When	the homepage finishes loading
Then	the visitor sees Featured Products and Trending Products, and the Recently Viewed and Personalized Recommendations rails are gracefully hidden or replaced with a general 'Popular Right Now' fallback rather than an empty section.

MOD-01-002 — Returning visitor with history

Condition	Description
Given	a customer has viewed products in a previous session on the same device
When	the customer returns to the homepage
Then	the Recently Viewed rail displays those products in most-recent-first order, and the Personalized Recommendations rail reflects the categories and price range of that browsing history.

5.2 Product Discovery (MOD-02)

Required Capabilities

- Responsive product grid (2 columns mobile, up to 4–5 columns desktop)
- Category browsing pages
- Filter panel: price range slider, minimum rating, availability (in stock/out of stock), discount/on-sale toggle, brand/category multi-select
- Sort controls: relevance, price low-to-high, price high-to-low, rating, newest
- Pagination or infinite scroll (implementer's choice, documented in the architecture doc deliverable)
- Wishlist toggle on every product card
- Quick View modal showing key details without a full page navigation
- Product comparison: ability to select 2–4 products and view a side-by-side comparison table

MOD-02-001 — Combined filter and sort

Condition	Description
Given	a customer is browsing the footwear category
When	the customer sets a price range of ₹2,000–₹5,000, a minimum rating of 4, and sorts by price low-to-high
Then	the grid updates to show only footwear products matching all active filters, sorted correctly, the active filter chips are visible, and the result count updates.

MOD-02-002 — No results

Condition	Description
Given	a customer applies filters with no matching products
When	the grid would render zero items
Then	the discovery page shows a clear no-results state with the specific filters that produced it and a one-click 'Clear Filters' action.

5.3 Search (MOD-03)

- Debounced search input (300–400ms) to avoid excessive requests
- Live search suggestions dropdown (matching products and categories)
- Locally stored recent search history, with a clear-history action
- Popular/trending searches shown when the input is focused but empty
- Search results page reusing the discovery grid, filters, and sorting
- A distinct, helpful no-results state with spelling/alternative suggestions where feasible
- Search state reflected in the URL as query parameters so results are shareable and support browser back/forward navigation

MOD-03-001 — Debounced query

Condition	Description
Given	a customer is typing into the search bar

Condition	Description
When	fewer than 300ms have passed since the last keystroke
Then	no network request is fired; a request fires only once typing pauses, preventing redundant calls to the product API.

5.4 Product Detail Page (MOD-04)

- Image gallery with thumbnail navigation and zoom-on-hover or lightbox
- Title, brand, category, average rating with review count, price with discount strike-through where applicable
- Variant selection (e.g., size/color) where the dataset supports it, with unavailable variants visually disabled
- Stock availability indicator, quantity stepper bounded by available stock
- Add to Cart and Buy Now actions
- Wishlist toggle
- Description and specifications in a tabbed or accordion layout
- Delivery information block (simulated pin-code based estimate)
- Related Products, Similar Products, Frequently Bought Together, and Recommended rails, each sourced from a distinct recommendation strategy (Section 6)
- Review summary and full review list (Section 5.10)

MOD-04-001 — Out-of-stock variant

Condition	Description
Given	a customer opens a product with multiple variants
When	the selected variant has zero stock
Then	the Add to Cart action is disabled with an explanatory label, and in-stock variants remain selectable.

5.5 Cart (MOD-05)

- Add, remove, and update quantity for line items, respecting per-variant stock limits
- Subtotal, simulated shipping calculation, simulated tax calculation, and grand total
- Coupon/discount code input with a small set of valid mock codes and clear invalid/expired states
- Save for Later — move an item out of the active cart without deleting it
- Recommended add-on products rail inside the cart (Cart-Based recommendation strategy)

Required Failure Handling

- Item goes out of stock while in the cart — flagged in place, checkout blocked until resolved
- Customer enters/selects a quantity greater than available stock — clamped with a message, never silently accepted
- A product in the cart is no longer returned by the product API — shown as unavailable with a Remove action, not silently dropped
- A product's price has changed since it was added — the customer is shown the new price and asked to confirm before proceeding
- The product API call fails — the cart falls back to last-known cached line-item data and shows a retry banner rather than a blank screen

MOD-05-001 — Stock validation on quantity change

Condition	Description
Given	a cart line item has 3 units in stock
When	the customer increases the quantity to 5
Then	the quantity is clamped to 3, an inline message explains the limit, and the subtotal reflects the corrected quantity.

5.6 Checkout (MOD-06)

Checkout is a realistic, multi-step simulated flow. No real payment gateway, real payment credentials, or real personal financial data is used anywhere in this flow.

1. Customer Information — name, email, phone, validated inline
2. Address — shipping address form with basic validation and a saved-address option for registered customers
3. Delivery Method — 2–3 simulated delivery speed options with simulated cost/ETA differences
4. Order Summary — full line-item, discount, shipping, and tax breakdown before payment
5. Mock Payment — a clearly labeled simulated payment step (e.g., 'Mock Card', 'Mock UPI', 'Mock Cash on Delivery') that performs client-side validation only and never transmits data anywhere
6. Confirmation — an order confirmation screen with a generated order ID and a summary, followed by an entry in Order History

> **Payment Flow Environment**
>
> The checkout flow operates in a sandboxed payment environment and must not include any field labeled or structured to resemble a request for a real card number, CVV, bank login, or UPI PIN. All payment fields are clearly marked as test-mode so the flow cannot be mistaken for a live payment form.

MOD-06-001 — Session expiry during checkout

Condition	Description
Given	a customer has reached the payment step of checkout
When	their session expires due to inactivity
Then	the application saves the in-progress checkout state locally, prompts re-authentication, and restores the customer to the same step after successful login rather than restarting checkout from scratch.

MOD-06-002 — Refresh mid-checkout

Condition	Description
Given	a customer is on the Address step of checkout
When	the customer refreshes the browser
Then	the checkout progress and entered data are restored from local persistence rather than being lost.

5.7 User Dashboard (MOD-07)

- Profile — view and edit name, email, phone, avatar (local only)
- Addresses — add, edit, delete, and mark a default shipping address
- Orders — order list with status, date, and total; order detail drill-down
- Wishlist — full wishlist management (see 5.9)
- Recently Viewed — full history, not just the homepage rail preview
- Reviews — reviews the customer has submitted, editable/removable
- Recommendation Preferences — a transparency panel letting the customer see which signals are influencing their recommendations and reset them

> **Personalization Transparency**
>
> Because the recommendation engine is signal-based rather than a black box, the dashboard must let the customer see, in plain language, what it is using: for example, 'Because you viewed: Running Shoes, Wireless Earbuds' and 'Your typical price range: ₹1,500–₹6,000'. This builds trust and also doubles as a demonstration of the recommendation architecture during evaluation.

5.8 Order Tracking (MOD-08)

Every simulated order progresses through a fixed set of states, rendered as a horizontal or vertical timeline:

Order Placed → Confirmed → Packed → Shipped → Out for Delivery → Delivered

- Order ID, ordered products with quantity and price, delivery address, expected delivery date
- Current status highlighted on the timeline with a timestamp for each completed stage
- Status progression may be simulated (e.g., advanced automatically over time or via an admin action) since no real logistics system exists

MOD-08-001 — Viewing an in-progress order

Condition	Description
Given	a customer has an order in the 'Shipped' state
When	the customer opens Order Tracking for that order
Then	the timeline shows Order Placed, Confirmed, Packed, and Shipped as completed with timestamps, and Out for Delivery and Delivered as upcoming.

5.9 Wishlist (MOD-09)

- Add and remove products from any product card or the product detail page
- Move a wishlist item directly into the cart
- Show current price and stock status for each wishlisted item, since these can change over time
- Wishlist-derived recommendations rail ('Because of your Wishlist')

MOD-09-001 — Wishlisted item goes out of stock

Condition	Description
Given	a product in a customer's wishlist becomes out of stock
When	the customer views their wishlist
Then	that item is visually marked out of stock and 'Move to Cart' is disabled for it, without removing it from the list.

5.10 Reviews (MOD-10)

- Star rating (1–5) plus a written review with a minimum/maximum character policy
- Review listing with sort (most recent, highest rated, lowest rated, most helpful)
- Rating distribution bar chart (5-star through 1-star breakdown)
- Helpful / Not Helpful voting per review
- A verified-purchase visual indicator

> **Verified Purchase — Data Source Note**
>
> 'Verified purchase' status is derived from the customer's local order history rather than a payment-provider record, since order fulfilment is handled by the application's own data layer rather than an external fulfilment system. The underlying logic and internal documentation should describe this data source clearly for engineering handoff.

6. AI Recommendation System

The recommendation engine is the centerpiece of ShopSphere AI and the primary differentiator from a plain product-listing clone. It must never present random or arbitrary products as 'recommended.' Every recommendation must be traceable to a specific, explainable signal or combination of signals.

6.1 Recommendation Signals

The engine draws on any of the following signals that are available for a given customer or session:

Signal	Source	Example Use
Recently viewed products	Local/session history	Surface similar or complementary items
Search history	Local storage	Infer interest categories
Product category & subcategory	Product dataset	Category-based recommendations
Product tags	Product dataset	Tag-similarity scoring
Price range	Derived from browsing/cart behavior	Price-based recommendations
Product rating	Product dataset	Quality-weighted ranking
Wishlist contents	Local/account storage	Wishlist-based recommendations
Cart contents	Local/account storage	Cart-based and bundle recommendations
Previous (simulated) purchases	Local order history	Repeat/replenishment suggestions
Brand preference	Derived from browsing frequency	Brand-affinity recommendations

6.2 Recommendation Scoring Model

Recommendations are ranked using a weighted composite score. The exact weights are an implementation decision for the engineering team delivering this project, but the score must be a function of, at minimum, the components below, and the composition must be documented in the technical README so it is auditable:

```
score(product) =
    w1 * categoryMatch(product, signals)
  + w2 * priceSimilarity(product, signals)
  + w3 * tagOverlap(product, signals)
  + w4 * normalizedRating(product)
  + w5 * recentlyViewedRelation(product, signals)
  + w6 * wishlistRelation(product, signals)
  + w7 * cartRelation(product, signals)

// Each component is normalized to a 0-1 range before weighting
// so no single signal can dominate the score unintentionally.
```

Worked example: a customer views a pair of running shoes tagged footwear / running, priced at ₹3,200 with a 4.4 rating. The engine records category = footwear, subcategory = running, priceBand = ₹2,000–₹5,000. When scoring candidates, a running backpack tagged running with a similar price and a 4.3 rating scores highly on categoryMatch, tagOverlap, and priceSimilarity even though it is a different product type — which is the intended 'frequently associated' behavior a shopper expects from a smart storefront.

6.3 Recommendation Strategies

#	Strategy	Primary Signal(s)	Typical Placement
1	Recently Viewed	Session/local view history	Homepage, header dropdown
2	Similar Products	Category + tags + price of current product	Product detail page
3	Category-Based	Current or dominant browsed category	Category pages, homepage
4	Price-Based	Customer's typical price band	Homepage, search results
5	Wishlist-Based	Wishlist contents	Wishlist page, homepage
6	Cart-Based	Current cart contents	Cart page, checkout summary
7	Frequently Bought Together	Co-occurrence in mock order/cart data	Product detail page
8	Trending Products	Mock popularity/rating velocity	Homepage
9	Personalized For You	Weighted blend of all available signals	Homepage, dashboard

The UI must label each rail with a signal-transparent heading rather than a generic one — for example, 'Recommended For You', 'Because You Viewed Running Shoes', 'Similar Products', and 'You May Also Like' — so the personalization is visible and legible to the shopper, not just present in the background.

6.4 Recommendation Service Architecture

Recommendation logic must be isolated from UI components behind a clean layered abstraction. No React component may contain scoring or filtering logic directly.

```
UI Components (rails, cards, badges)
        |
Recommendation Hook            e.g. useRecommendations()
        |
Recommendation Service         orchestration, caching, signal assembly
        |
Recommendation Provider        interface / contract boundary
        |
Mock Recommendation Engine     scoring + strategy implementations
        |
Product Dataset / Mock API     DummyJSON + local mock data
```

Conceptual service endpoints exposed by the Recommendation Service layer (implemented as frontend service functions, not necessarily real network endpoints):

```
GET /recommendations/personalized
GET /recommendations/similar/:productId
GET /recommendations/trending
GET /recommendations/frequently-bought/:productId
```

Why the Abstraction Matters

The Recommendation Provider boundary is what allows this system to evolve. Today the Provider is backed by the Mock Recommendation Engine described above. As the platform scales, the same Recommendation Service could call an ML/LLM-backed provider without any change to the UI layer. Reviewers will specifically check that this separation of concerns exists and is not just conceptual — it must be reflected in the actual folder and module structure.

7. AI Shopping Assistant

7.1 Concept

The AI Shopping Assistant is a chat-style interface that lets a customer describe what they want in natural, conversational language and receive a filtered, relevant set of products in response. It is explicitly a Rule-Based / Mock AI Shopping Assistant: it uses structured intent parsing (keyword, pattern, and entity extraction) rather than a real language model, and this must be stated plainly in any user-facing About/Help copy as well as in the technical documentation.

7.2 Required Interface Elements

- Persistent chat interface (panel or full-screen on mobile) with message history for the session
- Distinct visual treatment for user messages versus assistant messages
- Inline product cards rendered directly in the conversation when the assistant returns results
- Conversation context retained across turns within a session (see 7.4)
- A row of suggested starter prompts for first-time use
- Loading indicator while intent is being parsed and results assembled
- A graceful fallback message when intent cannot be confidently parsed, with clarifying suggestions

7.3 Structured Intent Extraction

The assistant must convert free-text input into a structured filter object before querying the product dataset. Example:

```
User Input:
  "Show me smartphones under 30,000 rupees with good ratings"

Structured Intent:
  {
    category: "smartphones",
    maxPrice: 30000,
    minRating: 4
  }
```

The Recommendation Service then executes this structured intent against the product dataset using the same filtering and scoring infrastructure used elsewhere in the application — the assistant is a new entry point into existing services, not a parallel implementation.

7.4 Conversational Refinement

The assistant must support at least the following categories of follow-up refinement against the existing conversation context, without requiring the customer to repeat the full query:

Follow-up Utterance	Expected Behavior
"Show me something cheaper"	Reduce maxPrice relative to the last result set (e.g., by a defined percentage or to the median of currently shown results) and re-query
"Something with better ratings"	Raise minRating and re-sort/re-filter the existing category/price context

Follow-up Utterance	Expected Behavior
"Only in blue" / variant-style refinement	Apply an additional attribute filter on top of existing intent, where the dataset supports it
"Show me more like the second one"	Trigger the Similar Products strategy using the referenced product as the seed
"Never mind, show me headphones instead"	Recognize a topic change and reset category-specific context while preserving general preferences (price band) if appropriate

7.5 Minimum Supported Intents

- Category + price ceiling: "Show me headphones under ₹5,000"
- Category + rating floor: "Show me smartphones with good ratings"
- Category + price + rating combined
- Comparative refinement: cheaper / more expensive / better rated
- Similarity requests referencing a previously shown product
- Unrecognized/ambiguous input handled with a clarifying question rather than a silent failure

MOD-AI-001 — Combined structured intent

Condition	Description
Given	a customer opens the AI Shopping Assistant
When	the customer types "Show me headphones under 5000"
Then	the assistant parses category = headphones and maxPrice = 5000, displays matching product cards inline, and states in plain language what filters were applied.

MOD-AI-002 — Refinement in context

Condition	Description
Given	the assistant has just shown headphone results under ₹5,000
When	the customer types "show me something with better ratings"
Then	the assistant keeps category = headphones and maxPrice = 5000, raises the rating threshold, and returns a refined, re-ranked result set without asking the customer to restate the category or price.

MOD-AI-003 — Unparseable input

Condition	Description
Given	a customer types an unrelated or unclear message such as "hmm not sure"
When	the assistant cannot extract a confident structured intent

Condition	Description
Then	the assistant responds with a clarifying message and 2–3 suggested prompts, and does not silently return unrelated products.

7.6 Optional Future Integration — Real AI/LLM

> **Not Required for Full Marks**
>
> A section describing optional future integration with a large language model provider is included for architectural completeness only. No API key is provisioned for this as part of the current build phase, and none should be placed in frontend/client-side code. This optional integration has no bearing on project evaluation. A correctly architected mock/rule-based implementation, as specified above, is eligible for full credit under the evaluation rubric in Section 19.

If an engineer chooses to explore this independently and outside the scope of graded requirements, the correct integration point is the Recommendation Provider / Assistant Provider boundary described in Section 6.4 — a real provider would implement the same interface contract as the Mock Recommendation Engine, keeping the UI and Service layers completely unchanged.

8. Admin Dashboard

A frontend-only administrative surface intended to demonstrate real operational tooling patterns: overview metrics, catalogue and order management, and recommendation analytics. It is gated behind a distinct admin role in the mock authentication layer (Section 11).

8.1 Dashboard Overview

- Total products, total orders, total customers (from mock/local data)
- Simulated revenue and simulated conversion rate, explicitly labeled as simulated
- Top-performing products (by simulated sales or by rating)
- Low-stock product alerts
- Recommendation engine click-through summary

8.2 Product Management

- Searchable, filterable product list reflecting the mock product dataset
- Product detail/edit view
- Add / edit / delete actions, simulated and scoped to local application state
- Category management (create/rename/assign categories locally)
- Inventory view with stock levels and low-stock indicators

8.3 Order Management

- Order list with filter by status, customer, and date range
- Order detail view with line items, customer, and totals
- Ability to simulate advancing an order's status along the tracking timeline defined in Section 5.8

8.4 Recommendation Analytics

This panel demonstrates that the recommendation architecture is not just functional but observable — a key expectation of a real product team.

Metric	Definition
Recommendation Impressions	Count of times a recommendation rail/card was rendered to a customer, tracked locally
Recommendation Clicks	Count of times a customer clicked a recommended product
Recommendation Conversion	Simulated rate of clicked recommendations that led to an add-to-cart action
Popular Recommendation Types	Breakdown of which of the nine strategies (Section 6.3) generated the most engagement

> **Analytics Data Scope**

> Every metric on this panel is derived from local/session data rather than an external analytics pipeline. Admin screens showing these figures should carry a small 'Local Analytics' label so the data source is clear to anyone reviewing the dashboard.

9. Frontend Architecture

9.1 Architectural Principles

- Feature-based organization over type-based organization, so each domain (cart, recommendations, ai-assistant, etc.) is self-contained
- Strict separation between UI components, business/service logic, and data-fetching logic
- A single, consistent state-management approach — global client state in Zustand, server/cache state in TanStack Query — never mixed
- No component reaches directly into a fetch call or into recommendation scoring logic; both go through a service layer

9.2 Recommended Project Structure

```
src/
  app/                    Next.js App Router routes
  components/             Shared, reusable UI primitives
  features/
    auth/
    products/
    search/
    cart/
    wishlist/
    checkout/
    orders/
    recommendations/
    ai-assistant/
    admin/
  hooks/                  Cross-feature reusable hooks
  services/               API clients, recommendation service, assistant service
  lib/                    Framework-agnostic helpers, formatters, constants
  store/                  Zustand stores
  types/                  Shared TypeScript types (or JSDoc typedefs)
  utils/                  Pure utility functions
```

9.3 Layer Responsibilities

Layer	Responsibility	Must Not Do
components/	Presentational UI, layout, accessibility	Fetch data or contain business logic
features/*	Feature-scoped components, hooks, and local logic	Reach into another feature's internals directly
services/	API calls, recommendation scoring, intent parsing	Import React or render UI
store/	Global client state (cart, wishlist, UI state)	Own server/cache data better suited to TanStack Query
hooks/	Reusable stateful logic bridging services and components	Duplicate logic already in services/

9.4 Language Choice

The Implementation Engineer selects one language for the entire project: JavaScript or TypeScript. Both are equally acceptable and evaluated on the same rubric criteria; the project does not require using both. If TypeScript is selected, domain types (Product, CartItem, RecommendationSignal, Order, etc.) must be properly defined and reused, not scattered as inline 'any' types. If JavaScript is selected, equivalent clarity must be achieved through JSDoc annotations and consistent object shape conventions.

10. API and Data Layer

10.1 Primary Data Source

DummyJSON is used as the primary free mock product and commerce data API, providing product, category, search, pagination, user, cart, and simulated-authentication endpoints suitable for frontend development without requiring a paid account.

> **Data Layer Scope**
> DummyJSON's write operations (create, update, delete) are handled by the service itself and are not persisted across requests. This PRD treats DummyJSON as a development-stage data source that stands in for a production database during this engagement. Any write flow in ShopSphere AI (checkout, admin edits, reviews) therefore persists locally (application state or browser storage) for the current session, with a full backend data layer as the natural next step beyond this engagement.

10.2 API Integration Coverage

Domain	Source	Notes
Products	DummyJSON /products	List, detail, pagination, filtering
Categories	DummyJSON /products/categories	Category navigation and filters
Search	DummyJSON /products/search	Debounced query search
Users	DummyJSON /users	Mock user profile data
Cart	DummyJSON /carts (dev reference) + local persistence	Cart truth lives client-side
Authentication	DummyJSON /auth (dev reference) + local mock session	See Section 11
Recommendations	Local Recommendation Service (Section 6.4)	Not a DummyJSON endpoint
Orders, Reviews, Analytics	Local mock JSON + local persistence	See Section 10.3

10.3 Local Mock Data Requirement

Where the free product API does not provide sufficient or appropriately shaped data, the engineer must create local mock JSON fixtures rather than leaving the feature unbuilt or fabricating a paid backend dependency. Minimum expected fixture files:

- orders.json — sample order records covering every tracking status
- reviews.json — sample reviews with ratings, verified-purchase flags, and helpful counts
- recommendations.json — seed data supporting frequently-bought-together and trending strategies where live co-occurrence data isn't available
- user-preferences.json — structure for storing derived signals such as price band and preferred categories
- analytics.json — seed structure for the admin recommendation-analytics panel

The project must remain fully runnable end-to-end using only DummyJSON plus these local fixtures — no additional signups, keys, or paid services.

11. Authentication

11.1 Required Flows

- Registration (mock — creates a local user record)
- Login (validated against DummyJSON's simulated auth endpoint and/or a local mock user store)
- Logout, clearing session state
- Protected routes (checkout, dashboard, orders, admin) that redirect unauthenticated users to login
- Persisted session state across a page refresh (e.g., token or session flag in browser storage)
- Role-based UI — the Administrator role reveals the admin dashboard entry point; the Customer role does not see it

11.2 Architecture

The authentication layer should be structured as if a JWT-based backend exists — issuing a token-like session object on login, attaching it to subsequent authenticated calls, and expiring it — even though the implementation is a mock/local mechanism. This keeps the client code compatible with a future real authentication provider.

> **Authentication — Environment Scope**
>
> This project's authentication layer is documented as a development-stage implementation. It does not yet implement full production-grade concerns such as server-side password hashing against a managed identity provider, refresh-token rotation at scale, or credential-stuffing protection, since those depend on backend infrastructure outside this engagement's scope. No paid authentication provider (Auth0, Clerk paid tiers, Firebase paid tiers, etc.) is required or expected.

12. Performance Requirements

- Route-level lazy loading and dynamic imports for non-critical UI (modals, admin dashboard, assistant panel)
- Image optimization via Next.js Image component (proper sizing, lazy loading below the fold)
- Debounced search input to minimize redundant API calls
- Efficient, deduplicated API requests via TanStack Query's caching and stale-time configuration
- Pagination or infinite scroll on all large lists to avoid rendering unbounded DOM trees
- Memoization (useMemo/useCallback, React.memo where justified) to avoid unnecessary re-renders in list-heavy views
- Skeleton loading states instead of layout-shifting spinners on content-heavy pages
- Baseline awareness of Core Web Vitals (LCP, CLS, INP) with reasoning documented for any deliberate trade-off

13. Security Requirements

Even as a frontend-only project, ShopSphere AI must be built with the security discipline expected of production frontend code.

- No secrets, tokens, or credentials committed to GitHub, including in commit history
- Environment variables used for any configurable endpoint or flag, with a committed .env.example and a git-ignored .env.local
- No API keys of any kind present in client-side/browser-executed code
- Input validation on every form (registration, login, checkout, reviews, admin edit forms)
- Safe rendering of user-generated content (reviews, profile fields) — no raw HTML injection
- General XSS awareness: sanitizing or escaping any dynamic content rendered from user or API input
- Secure-by-convention authentication architecture as described in Section 11
- Protected routes enforced both by UI guarding and by graceful handling if a protected route is reached without a valid session
- Consistent, user-safe error handling that never leaks raw error objects or stack traces into the UI

14. Accessibility Requirements

ShopSphere AI should follow WCAG 2.2 Level AA principles wherever practical for a project of this scope.

- Semantic HTML landmarks (header, nav, main, footer) and correct heading hierarchy
- Full keyboard navigability, including the product grid, filters, cart, checkout, and the AI assistant chat
- Visible focus states on every interactive element
- Accessible forms — associated labels, inline error messaging tied to fields via aria-describedby
- Screen reader considerations for dynamic content (e.g., cart count updates, assistant responses) via appropriate live regions
- ARIA attributes applied where semantic HTML alone is insufficient (custom dropdowns, modals, tabs)
- Sufficient color contrast across text, buttons, and status badges
- Accessible modals — focus trapping, escape-to-close, and return-focus on close
- Accessible, non-color-only error and validation messaging

15. Responsive Design Requirements

ShopSphere AI must work as a genuinely designed experience — not a scaled-down desktop layout — across the following breakpoints:

Breakpoint	Approx. Width	Key Layout Behavior
Mobile	< 640px	Single-column grid, bottom-accessible cart/assistant access, collapsed filters in a drawer/sheet
Tablet	640–1024px	2–3 column grid, condensed navigation, filters as a side drawer or inline collapsible panel
Laptop	1024–1440px	3–4 column grid, persistent filter sidebar, full navigation
Desktop	1440–1920px	4–5 column grid, expanded hero and recommendation rails
Large Desktop	> 1920px	Max-width content container to preserve readability and prevent excessive line/image stretching

Navigation, filters, the cart, checkout, the AI assistant, and the admin dashboard must all be independently verified at each breakpoint — not just the homepage.

16. Loading, Error, and Empty States

Every major module listed in Section 5 must implement all four of the following states before it is considered complete. This is enforced by the Definition of Done in Section 20.

State	Requirement
Loading	Skeleton UI matching the eventual content's layout wherever practical, not a generic spinner for content-heavy views
Empty	A specific, useful message plus a clear next action (e.g., empty wishlist → 'Browse Products' button)
Error	A clear, human-readable message plus a recovery action (retry, go back, contact support placeholder)
Network Failure	A distinguishable offline/failure state, with cached data shown where available rather than a blank screen

16.1 Modules Requiring Explicit Failure-Path Coverage

- Product API failure (discovery grid, PDP, homepage rails)
- Search failure or timeout
- Cart failure (see Section 5.5 failure handling)
- Recommendation service failure — rails hide gracefully or fall back to Trending, never show a broken/empty box
- AI assistant failure — chat shows a clear inline error bubble with a retry option, conversation history is preserved
- Authentication failure — invalid credentials, expired session
- Mock checkout failure — simulated payment failure state with a retry path back to the payment step, not a full checkout restart

17. Testing Strategy

17.1 Unit Testing — Vitest

- Utility functions (formatters, validators, price/discount calculators)
- Recommendation scoring functions — each scoring component tested in isolation with known inputs/outputs
- Structured intent parsing logic for the AI assistant
- Custom hooks (with @testing-library/react-hooks patterns or equivalent)

17.2 Component Testing — React Testing Library

- Product card, filter panel, and search bar behavior
- Forms: registration, login, checkout steps, review submission — including validation error paths
- Cart interactions: add, remove, quantity change, stock-limit clamping
- Recommendation rail rendering, including empty/fallback states

17.3 End-to-End Testing — Playwright

- Registration and login
- Search and filter flow
- Product detail view and add-to-cart
- Wishlist add/remove and move-to-cart
- Full checkout flow through to order confirmation
- Order tracking view
- Recommendation rail interaction (click-through to product)
- AI shopping assistant conversation, including at least one refinement follow-up

17.4 Minimum Coverage Expectation

Every module in Section 5 should have at least one meaningful automated test exercising its primary happy path, and the recommendation scoring logic and AI intent parser specifically should have dedicated unit test suites given their central role in the product. A documented Testing Report deliverable (Section 21) should summarize coverage and any known gaps.

18. Version Control and Collaboration Practice

18.1 Repository Setup

- A single GitHub repository for the project, initialized with a proper .gitignore for the chosen stack
- A committed .env.example documenting required environment variables without real secret values

18.2 Branch Strategy

Even as a solo contributor, feature branches and pull requests should be used to simulate a real team workflow and produce a reviewable history:

```
main                          protected, always deployable
feature/project-setup
feature/product-search
feature/product-details
feature/cart
feature/checkout
feature/recommendations
feature/ai-assistant
feature/admin-dashboard
feature/testing-and-polish
```

18.3 Commit Conventions

Conventional, descriptive commit messages are required (e.g., feat: add price-based recommendation strategy, fix: correct stock clamping on cart quantity change, test: add unit tests for intent parser). Commit history should read as a legible engineering log, not a series of 'update' or 'wip' commits.

18.4 Issue Tracking

GitHub Issues (or an equivalent free board) should be used to track the phases in Section 19 as trackable units of work, giving reviewers visibility into planning and progress, not just the final code.

19. Development Plan — 4-Week Delivery Schedule

This project is scoped for a single Implementation Engineer working full-time across four weeks. The ten workstreams below are sequenced into four weekly milestones. Each phase lists objectives, key tasks, the expected output, and acceptance criteria that must be satisfied before moving on. The plan assumes standard focused effort and leaves internal buffer for debugging and polish within each week rather than at the very end only.

19.1 Week 1 — Foundation, Authentication, and Product Discovery

PHASE-01 — Project Setup and Architecture (Week 1, Days 1–2)

Objectives:

- Establish the codebase, tooling, and architecture that every later phase depends on.

Key Tasks:

- Initialize the Next.js project with the chosen language (JS or TS)
- Configure Tailwind CSS and install shadcn/ui components
- Set up the folder structure defined in Section 9.2
- Configure Zustand and TanStack Query providers
- Configure Vitest, React Testing Library, and Playwright
- Set up GitHub repository, branch strategy, and .env.example

Expected Output:

A running, empty-shell application with the full architecture, tooling, and CI-ready test runners in place, pushed to GitHub on main.

Acceptance Criteria:

- Project builds and runs locally with no console errors
- Folder structure matches Section 9.2
- A placeholder test runs successfully in each of the three test frameworks

PHASE-02 — Authentication (Week 1, Days 2–3)

Objectives:

- Deliver the Development Authentication flow described in Section 11.

Key Tasks:

- Build registration and login forms with validation
- Implement mock/JWT-compatible session handling and persistence
- Implement protected-route logic and role-based UI (Customer vs Administrator)

Expected Output:

A working login/registration flow that gates checkout, dashboard, and admin routes correctly.

Acceptance Criteria:

- Given valid demo-style credentials, login succeeds and session persists across refresh
- Given an unauthenticated visitor, protected routes redirect to login

PHASE-03 — Product Discovery (Week 1, Days 3–5)

Objectives:

- Deliver the product grid, filters, sorting, and search described in Sections 5.2–5.3.

Key Tasks:

- Integrate DummyJSON product and category endpoints
- Build the responsive product grid, filter panel, and sort controls
- Build debounced search with suggestions and URL query sync
- Implement loading, empty, and error states for discovery and search

Expected Output:

A fully functional, responsive discovery and search experience covering all states in Section 16.

Acceptance Criteria:

- MOD-02-001 and MOD-02-002 acceptance criteria pass
- MOD-03-001 debounce behavior verified

19.2 Week 2 — Product Detail, Cart, and Checkout

PHASE-04 — Product Detail Page (Week 2, Days 6–7)

Objectives:

- Deliver the full PDP experience described in Section 5.4.

Key Tasks:

- Build image gallery, variant selection, and stock-aware quantity controls
- Build description/specification tabs and delivery information block
- Wire up Related, Similar, and Frequently Bought Together rail placeholders (full logic lands in Week 3)

Expected Output:

A complete, responsive PDP with all static and stock-dependent behavior working.

Acceptance Criteria:

- MOD-04-001 acceptance criteria passes
- PDP is fully usable at all breakpoints in Section 15

PHASE-05 — Cart and Wishlist (Week 2, Days 7–9)

Objectives:

- Deliver cart and wishlist functionality and their required failure handling.

Key Tasks:

- Build cart state management (Zustand) with stock validation
- Implement coupon codes, save-for-later, subtotal/shipping/tax simulation
- Build wishlist add/remove/move-to-cart
- Implement all failure-path handling listed in Section 5.5

Expected Output:

A robust cart and wishlist that correctly handles every failure scenario specified in Section 5.5 and 5.9.

Acceptance Criteria:

- MOD-05-001 and MOD-09-001 acceptance criteria pass

PHASE-06 — Checkout and Orders (Week 2, Days 9–10)

Objectives:

- Deliver the simulated multi-step checkout and order history/tracking.

Key Tasks:

- Build the six-step checkout flow described in Section 5.6
- Implement mock payment simulation with client-side-only validation
- Build order history, order detail, and the tracking timeline in Section 5.8
- Implement checkout persistence across refresh and session expiry handling

Expected Output:

A complete, resilient checkout flow producing a real entry in order history and tracking.

Acceptance Criteria:

- MOD-06-001 and MOD-06-002 acceptance criteria pass
- MOD-08-001 acceptance criteria passes

19.3 Week 3 — Recommendation Engine and AI Shopping Assistant

PHASE-07 — Recommendation Engine (Week 3, Days 11–13)

Objectives:

- Deliver the full recommendation architecture and all nine strategies described in Section 6.

Key Tasks:

- Implement the layered architecture: hook → service → provider → mock engine
- Implement the weighted scoring model and all nine recommendation strategies
- Wire recommendation rails into the homepage, PDP, cart, and wishlist
- Build the Recommendation Preferences transparency panel in the user dashboard

Expected Output:

A working, explainable recommendation system visible across the product with correct signal-based reasoning.

Acceptance Criteria:

- MOD-01-002 acceptance criteria passes
- Each of the nine strategies produces visibly distinct, signal-appropriate results in manual testing

PHASE-08 — AI Shopping Assistant (Week 3, Days 13–15)

Objectives:

- Deliver the rule-based conversational assistant described in Section 7.

Key Tasks:

- Build the chat interface with inline product cards
- Implement structured intent extraction (category, price, rating)

- Implement conversational refinement handling (cheaper, better rated, similarity requests)
- Implement graceful fallback for unparseable input

Expected Output:

A working assistant satisfying the minimum supported intents and refinement behaviors in Section 7.5.

Acceptance Criteria:

- MOD-AI-001, MOD-AI-002, and MOD-AI-003 acceptance criteria pass

19.4 Week 4 — Admin Dashboard, Reviews, Quality, and Launch

PHASE-09 — Admin Dashboard and Reviews (Week 4, Days 16–18)

Objectives:

- Deliver the admin dashboard (Section 8) and the reviews module (Section 5.10).

Key Tasks:

- Build dashboard overview, product management, and order management screens
- Build recommendation analytics panel with clearly labeled simulated data
- Build review submission, listing, sorting, and rating distribution

Expected Output:

A complete, clearly-labeled-as-simulated admin experience and a fully functional reviews module.

Acceptance Criteria:

- All admin write actions are visually labeled as local/simulated per Section 4.2
- Review submission and listing work end-to-end

PHASE-10 — Testing, Performance, Accessibility, and Deployment (Week 4, Days 18–20)

Objectives:

- Close out quality gates and ship the deployed application.

Key Tasks:

- Complete unit, component, and end-to-end test suites per Section 17
- Complete performance pass per Section 12 (lazy loading, image optimization, memoization)
- Complete accessibility pass per Section 14 (keyboard nav, ARIA, contrast)
- Verify all states in Section 16 across all modules
- Deploy to a free-tier-compatible host and verify the production build
- Finalize README, architecture documentation, and all deliverables in Section 21

Expected Output:

A deployed, tested, accessible, and fully documented production build ready for evaluation.

Acceptance Criteria:

- Every item in the Definition of Done (Section 20) is satisfied
- The deployed URL is reachable and functionally equivalent to local development

19.5 Weekly Milestone Summary

Week	Milestone	Primary Risk to Manage
Week 1	Architecture, auth, and discovery are functional	Under-scoping the folder/service architecture early causes rework later
Week 2	Full commerce loop (browse → cart → checkout → order) works end-to-end	Cart/stock edge cases are easy to under-test — validate against Section 5.5 explicitly
Week 3	Recommendation engine and AI assistant are live across the product	Keeping scoring logic out of UI components requires discipline — review against Section 6.4
Week 4	Admin, reviews, quality gates, and deployment are complete	Reserve real time for accessibility and testing — do not treat them as final-day cleanup

20. Edge Cases and Expected Behavior

The following edge cases must be deliberately handled, not discovered accidentally during evaluation. Each row states the expected frontend behavior.

#	Edge Case	Expected Behavior
1	Product becomes unavailable after being added to cart	Flagged in cart with a clear label; checkout blocked until removed or resolved
2	Product removed from the API between page loads	Product detail shows a 'no longer available' state, not a crash or blank page
3	Product API request fails	Cached/last-known data shown where possible with a retry banner
4	Slow network / high latency	Skeleton loading states rather than an indefinite blank screen
5	Empty search input submitted	No request fired; recent/popular searches shown instead
6	Search returns no results	Distinct no-results state with suggestions and a clear-filters action
7	Invalid filter combination (e.g., min price > max price)	Inline correction or automatic swap, never a broken query
8	Duplicate item added to cart	Quantity increments on the existing line item rather than creating a duplicate row
9	Out-of-stock product viewed	Add to Cart disabled with explanation; notify-me placeholder acceptable
10	Price change on a cart item	Customer shown old vs. new price and asked to confirm before proceeding
11	Invalid or expired coupon code	Inline error explaining why the code was rejected
12	Empty wishlist viewed	Empty state with a 'Browse Products' call to action
13	Empty order history viewed	Empty state encouraging first purchase, not a blank table
14	Simulated payment fails	Clear failure state with a retry path back to the payment step only
15	AI assistant receives an unparseable query	Clarifying fallback message with suggested prompts, per MOD-AI-003
16	Recommendation service returns no candidates	Rail hides gracefully or falls back to Trending Products, never renders empty boxes
17	User refreshes mid-checkout	Checkout state restored from local persistence, per MOD-06-002
18	Session expires mid-checkout	State preserved and restored post re-authentication, per MOD-06-001
19	Invalid login credentials	Clear inline error, no ambiguity about which field failed
20	Product with a missing or broken image	Graceful placeholder image, never a broken-image icon
21	Malformed or unexpected API response shape	Defensive parsing with a fallback UI rather than a runtime crash

#	Edge Case	Expected Behavior
22	Extremely long search query	Input and request handled without layout breakage or request failure
23	Recommendation includes an out-of-stock or removed product	Filtered out before rendering, or visibly marked unavailable within the rail
24	New user with no browsing history at all	Personalized rails fall back to Trending/Featured per MOD-01-001
25	User has insufficient signals for confident personalization (e.g., only one page view)	Engine blends available signals with general popularity rather than over-fitting to a single data point
26	Two browser tabs modify the cart simultaneously	Last-write consistency maintained without duplicated or lost line items on reload
27	Admin attempts to delete a product referenced by an existing simulated order	Deletion blocked or the product is flagged as archived rather than silently breaking order history

21. Test Data Requirements

Test data is local to each engineer's own implementation. The engineer is responsible for creating their own local development accounts and sample data sets sufficient to exercise every flow in this PRD, including at least one Customer-role account and one Administrator-role account for manual and automated testing.

> **Test Data Guidance**
>
> Any account created for development and testing purposes should be a local account used only within the development environment. Personal information, real email accounts tied to sensitive data, and real payment details should never be entered anywhere in this project.

22. Engineering Deliverables

The following deliverables constitute a complete submission. No paid resource is required to produce any of them.

#	Deliverable	Description
1	GitHub Repository	Full source history reflecting the branch strategy in Section 18.2
2	Live Deployment URL	Working production build on a free-tier-compatible host
3	README.md	Setup instructions, environment variables, scripts, and a project summary
4	Architecture Documentation	Explanation of the folder structure, state management approach, and layer responsibilities (Section 9)
5	API Integration Documentation	Which endpoints are used, which are mocked, and why (Section 10)
6	Testing Report	Summary of unit, component, and end-to-end coverage with known gaps noted
7	Screenshots	Key screens across at least three breakpoints
8	Demo Video	A short, narrated walkthrough of the core flows, including the recommendation engine and AI assistant
9	Final Presentation	A concise walkthrough for mentor review covering scope, architecture, and trade-offs made
10	Final Technical Report	Reflection on decisions, challenges, and what a production version would add (ties into Section 23)

23. Definition of Done

A feature from Section 5 onward is only considered complete when every item below is true for it:

- Functional requirements for the feature are fully implemented
- Responsive UI verified across all breakpoints in Section 15
- Loading, error, and empty states implemented per Section 16
- Relevant API/service integration works against real DummyJSON data and/or local mock data
- Form and input validation implemented where applicable
- Accessibility considerations from Section 14 addressed
- At least one automated test exists covering the primary happy path
- No critical or unresolved console errors remain
- Code is committed to the correct feature branch with descriptive commits and merged via pull request
- README and/or architecture documentation updated to reflect the feature
- Feature verified working in the deployed build, not only in local development

24. Evaluation Rubric

ShopSphere AI is evaluated out of 100 marks across the categories below. No marks are awarded for the use of any paid API, subscription, or service anywhere in the project — a submission using only free/open-source resources is fully eligible for 100/100.

Category	Marks	Full Marks Criteria
UI/UX	15	Polished, consistent, production-feeling interface; thoughtful micro-interactions; clear information hierarchy
React/Next.js Architecture	15	Clean feature-based structure per Section 9; correct use of App Router, server/client boundaries, and component composition
JavaScript/TypeScript Quality	10	Consistent, readable, well-organized code; proper typing or JSDoc discipline; no dead code
API Integration	10	Correct, resilient integration with DummyJSON and local mock data per Section 10, including error handling
State Management	10	Clean separation of Zustand (client state) and TanStack Query (server/cache state) with no unnecessary duplication
E-Commerce Functionality	15	Discovery, PDP, cart, checkout, orders, wishlist, and reviews all fully functional per Section 5
Recommendation Engine	10	All nine strategies implemented, correctly scored, explainable, and architecturally isolated per Section 6
Testing	5	Meaningful unit, component, and end-to-end coverage per Section 17
Performance	5	Lazy loading, image optimization, caching, and memoization applied per Section 12
Accessibility & Security	3	WCAG-aligned practices per Section 14 and security discipline per Section 13
Git/GitHub Practices	2	Clean, descriptive commit history and proper use of feature branches and pull requests

24.1 Partial and Low-Mark Guidance

- Partial marks are awarded where a category is implemented but incomplete — e.g., recommendations work but only 4 of 9 strategies are distinguishable, or tests exist but do not cover the AI assistant or recommendation scoring.
- Low marks are awarded where a category is present only superficially — e.g., a single generic 'Recommended Products' rail with no signal-based logic, or a UI that visibly breaks at one or more required breakpoints.
- No category is scored based on the use of paid tools, and no category loses marks for correctly using a free/mock equivalent as specified in this PRD.

25. Platform Roadmap — Current and Future Architecture

This section exists so the engineering reasoning behind each interim architectural choice in this PRD is explicit and auditable — and so the Engineer understands precisely which layer of the architecture would need to change, and which would not, as the platform scales.

25.1 Current Build-Phase Architecture

```
Next.js
   |
Product API (DummyJSON)
   |
Recommendation Engine (rule-based)
   |
Authentication Layer (local/JWT-compatible)
   |
Checkout Flow (client-side)
```

25.2 Scaled Platform Architecture

```
Next.js
   |
Backend API
   |
Database
   |
Authentication Service
   |
Payment Gateway
   |
Recommendation Engine
   |
ML/LLM Service
   |
Analytics Infrastructure
```

25.3 What This Build Phase Delivers

The Engineer is building the frontend product architecture and the AI integration abstraction layer — the Recommendation Service/Provider boundary, the structured-intent assistant pipeline, the state and caching strategy, and the full commerce UI — establishing the interface contracts a backend and ML/LLM service would plug into as the platform scales. Every component in Section 25.1 sits behind an interface that the corresponding component in Section 25.2 could satisfy later without the UI layer being aware of the difference. Demonstrating that this boundary is real and enforced in the codebase — not just described in documentation — is one of the most heavily weighted signals in evaluation.

Appendix A — Zero-Cost Technology Stack

A.1 Language

The Implementation Engineer selects exactly one: JavaScript or TypeScript. Both are fully supported by every tool in this stack.

A.2 Mandatory Stack

Layer	Technology	Cost
Framework	React + Next.js	Free / Open Source
Markup & Styling Base	HTML5, CSS3	Free
Styling System	Tailwind CSS	Free / Open Source
UI Components	shadcn/ui	Free / Open Source
Global State	Zustand	Free / Open Source
Server/Cache State	TanStack Query	Free / Open Source
API	REST (DummyJSON + local mock endpoints)	Free
Authentication	Mock / JWT-compatible frontend architecture	Free
Version Control	Git + GitHub (Free tier)	Free
Unit Testing	Vitest	Free / Open Source
Component Testing	React Testing Library	Free / Open Source
End-to-End Testing	Playwright	Free / Open Source
Deployment	Free-tier-compatible host (e.g., Vercel Hobby tier)	Free, subject to current provider terms
Editor	Visual Studio Code	Free

> **Free-Tier Terms May Change**
>
> Recommended deployment platforms are referenced based on their free-tier offering at the time of writing. The Implementation Engineer should confirm current free-tier terms before deployment and select any comparable free-tier-compatible alternative if terms have changed, without needing PRD sign-off for that substitution.

26. Closing Notes

This PRD is intended to be read the way a frontend engineer reads a specification: end to end before starting, and then repeatedly as a reference during implementation. Where this document is silent on a specific interaction detail, the Engineer should make a reasonable, documented product decision consistent with the principles in Sections 1 and 9, rather than leaving the behavior undefined.

Questions about scope, ambiguity, or feasibility should be raised with product and engineering leadership early — the same way an engineer on any product team would raise them with a Product Manager or Engineering Lead — rather than resolved silently in a way that diverges from this specification.

— Product Management, UX Design, and Engineering Leadership